Applied Machine Learning

Time Series
Feature Engineering



Learning goals

- Calendar and cyclic features
- Lagged and rolling features
- Feature importance and selection

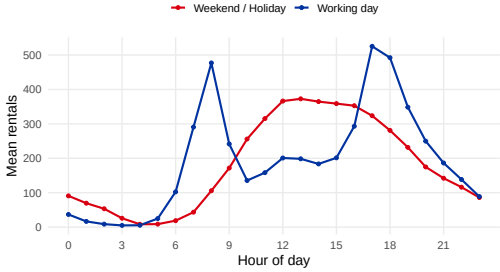
Feature Engineering



CALENDAR FEATURES I



- Day-of-Week, Day-of-Month, Week Number, Month, Quarter
- Boolean flags: public holidays, weekend, promotion days, end-of-month



Why?

Capture human or process-driven periodicity. E.g., bike rentals peak on weekday mornings (commuters) but are spread throughout the day on weekends.

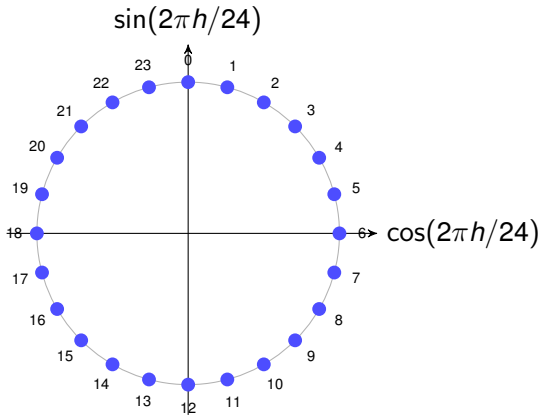
CYCLIC ENCODING I



$$\sin(2\pi \frac{hour}{24}),\ \cos(2\pi \frac{hour}{24})$$

- Converts periodic integers (hour 0-23, month 1-12) into continuous space
- Eliminates artificial jump between 23 $\rightarrow$ 0 (and December $\rightarrow$ January)
- Most beneficial for linear and distance-based models; trees can often learn periodicity from raw integers but may need deeper splits to do so

CYCLIC ENCODING VISUALIZATION (HOUR 0-23) I



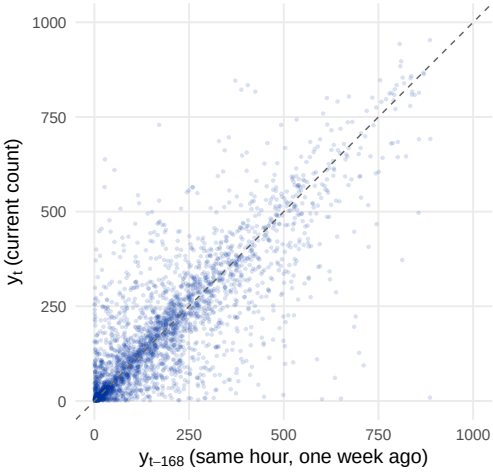
LAGGED & AUTO-REGRESSIVE FEATURES I



- Plain lags: y_{t-1}, y_{t-24}, y_{t-168} (1 hour ago, yesterday, last week)
- Choose lag periods that match the data's seasonality (e.g., 168h = 1 week for hourly data)
- Multi-step: include future-known covariates (e.g., planned price, weather forecast)
- Combine with calendar features for interactions

WHY DO LAGS WORK? I





y_t vs y_{t-168} (same hour, one week ago). The strong diagonal correlation shows that last week's count is a powerful predictor of this week's count.

ROLLING / EXPANDING STATISTICS I



Compute summary statistics over a sliding window of past observations:

- **Rolling mean / std**: $\bar{y}_{t,w} = \frac{1}{w} \sum_{i=t-w}^{t-1} y_i$ over window of size w
- **Rolling min / max**: capture recent extremes
- **Expanding mean**: $\bar{y}_t = \frac{1}{t-1} \sum_{i=1}^{t-1} y_i$ acts as a trend indicator
- **Percentiles**: 25th, 75th for distribution shape

Important: the window must only look **backward** (no future data!). When using lag features, shift the window by the forecasting horizon to avoid leakage.

INTERACTION FEATURES I



Combine temporal features with covariates to capture conditional effects:

- Lag $\times$ Holiday flag $\rightarrow$ “was the same hour last week also a holiday?”
- Weather $\times$ Weekend indicator $\rightarrow$ “good weather on weekends drives leisure rentals”
- Hour $\times$ Workingday $\rightarrow$ commuter peaks only on working days

Tree-based models can learn interactions automatically, but explicit features can help simpler models and improve interpretability.

FEATURE IMPORTANCE & SELECTION I



With many engineered features, check which ones actually help:

- **Permutation importance**: shuffle one feature at a time, measure the drop in validation performance
- **Tree-based (impurity) importance**: built-in `feature_importances_` attribute (fast, but biased toward high-cardinality features; permutation importance does not have this bias)
- **SHAP values**: consistent, model-agnostic feature attributions (more expensive to compute)

Rule of thumb: start with many features, inspect importance, prune correlated or weak ones.

See the Interpretable Machine Learning course for more details on feature importance and model interpretation methods:
`https://slds-lmu.github.io/iml/`

PUTTING IT ALL TOGETHER: RAW ROW → ML-READY ROW I



One row of the bike dataset **before** feature engineering:

holiday	temp	humidity	windspeed	datetime
False	24.6	0.45	0.30	2012-06-15 17:00

The same row **after** feature engineering:

holiday	temp	hour	dow*	hour_sin	hour_cos	lag_168	lag_336	roll_mean
False	24.6	17	4	−0.97	−0.26	432	389	285.3

The datetime column is gone. All temporal information is now encoded as numeric features that any standard ML model can consume.

*dow = day of week (Mon=0, Sun=6)